COMP3711 Midterm, the plain-words sheet (side 1 of 2). Fri Oct 16, 20:00, LTC. Closed book, this sheet allowed, no calculator.

1. What the symbols mean

Symbol	Read it as	Example
n	the input size (number of items in the array)	A[1..n] has n numbers
T(n)	running time (number of steps) on an input of size n	T(n) = 3n² + 5
O(g)	"at most g, up to a constant, for big n" (upper bound)	3n²+5 = O(n²), also O(n³)
Ω(g)	"at least g, up to a constant" (lower bound)	3n²+5 = Ω(n²), also Ω(n)
Θ(g)	"exactly g, up to a constant" = both O and Ω	3n²+5 = Θ(n²)
log n	log base 2: how many times you can halve n before reaching 1	log 8 = 3, log 1024 = 10
log² n	(log n)², i.e. log n times log n	log² 8 = 9
log log n	log of log n, tiny	loglog 65536 = 4
⌊x⌋, ⌈x⌉	round down, round up	⌊7/2⌋ = 3, ⌈7/2⌉ = 4
Σ_{i=1}^{n} f(i)	add f(1) + f(2) + … + f(n)	Σ i = 1+2+…+n = n(n+1)/2
n!	n factorial = n·(n–1)·…·1, the number of orderings of n items	4! = 24
C(n,k) or "n choose k"	number of ways to pick k items out of n	C(4,2) = 6
E[X]	expected (average) value of a random quantity X	E[die] = 3.5
Pr[A]	probability that A happens	Pr[coin heads] = ½
A[l..r]	the part of array A from position l to r	A[3..5] is 3 items
x ← 5	assign 5 to x (pseudocode)	

Which is faster? Smaller running time = faster. Order from fast to slow: 1 (constant) < log n < √n < n < n log n < n² < n³ < 2^n < n!. Any power of log n is slower-growing than any power of n (even n^0.01). Any polynomial is slower-growing than any exponential (even 1.01^n).

Rules you will actually use: keep only the biggest term, drop constants ($3n^2 + 100n \to n^2$). log of any base is the same up to a constant. log(n^c) = c·log n = Θ(log n). a^(log_b n) = n^(log_b a), e.g. $4^{(\log_2 n)} = n^2$. 2^(2n) = 4^n is NOT Θ(2^n) but 2^(n+2) = 4·2^n IS Θ(2^n). Sum of 1 + ½ + ⅓ + … + 1/n = Θ(log n). Sum of 1 + 2 + … + n = Θ(n²). Sum of 1 + ½ + ¼ + … = 2 = Θ(1).

2. The A / B / U table (always question 1)

You get T_A and T_B. Write **A** if A is always faster for big n, **B** if B is, **U** if you cannot tell. Steps: (1) simplify both to one term; (2) if both are Θ (exact) and different, the smaller one wins; (3) if both are the same Θ, write U; (4) if one is only O (upper bound) or only Ω (lower bound), ask "could the hidden true value flip the order?" If yes, U. If the bound still forces the order (e.g. O(n²) vs Ω(2^n): A is at most n², B at least 2^n, so A), write A or B. Two O's or two Ω's: U. A recurrence as T_B: solve it first.

3. Solving a recurrence (always asked)

A recurrence like T(n) = 2T(n/2) + n means "to solve size n, solve two halves, then do n extra work". **Method 1, Master table:** for T(n) = aT(n/b) + n^d compare a with b^d: a > b^d → Θ(n^(log_b a)); a = b^d → Θ(n^d log n); a < b^d → Θ(n^d). **Method 2, expansion (use when they say "from first principles"):**

```
T(n) = 2T(n/2) + n
     = 2[2T(n/4) + n/2] + n       = 4T(n/4) + 2n
     = 4[2T(n/8) + n/4] + 2n      = 8T(n/8) + 3n
     ... after k steps:            = 2^k T(n/2^k) + k·n
stop when n/2^k = 1, i.e. k = log n:
     = n·T(1) + n log n            = Θ(n log n)
```

Method 3, recursion tree: level i has a^i nodes, each of size n/b^i and cost f(n/b^i). Write the level total, add over levels 0..log_b n, add the leaves (n^(log_b a) of them, cost T(1) each). Three outcomes: every level equal → cost × number of levels; levels shrinking → the root wins; levels growing → the leaves win.

Recurrence	Answer	Why
T(n/2) + 1	Θ(log n)	binary search: 1 step per halving
2T(n/2) + n	Θ(n log n)	mergesort: n per level, log n levels
4T(n/2) + n	Θ(n²)	leaves win (4 > 2)
3T(n/2) + n	Θ(n^1.585)	Karatsuba, leaves win
7T(n/2) + n²	Θ(n^2.81)	Strassen
2T(n/2) + 1	Θ(n)	leaves win
T(n/2) + n	Θ(n)	root wins
2T(n–1) + 1	Θ(2^n)	doubles n times
T(n–1) + n	Θ(n²)	1 + 2 + … + n

4. How to write an algorithm answer

Always these five parts:
1. **Idea** in one or two sentences ("sort, then binary search twice").
2. **Name and I/O:** "Procedure Find(A, l, r, x). Input: sorted A[l..r] and value x. Output: position of x or nil."
3. **Steps** in pseudocode or numbered plain English. Use black boxes from class: "sort with mergesort, O(n log n)", "Select(A, k) from class, O(n)", "Merge, O(n)".
4. **Running time** with one line of reason: "each call does O(1) work and halves the range, so O(log n)" or "T(n) = 2T(n/2) + n = O(n log n)".
5. **Why correct**, one sentence (not a proof on this midterm).

```
Procedure Name(A, l, r)          // input, output stated above
  if l = r then return A[l]      // base case first
  m ← ⌊(l + r)/2⌋                // split
  x ← Name(A, l, m)              // recurse left
  y ← Name(A, m+1, r)            // recurse right
  return combine(x, y)           // combine, say exactly how
First call: Name(A, 1, n).  T(n) = 2T(n/2) + cost(combine).
```

Pseudocode style: if/then/else, for i ← 1 to n, while, return. Write "x = the maximum of A" in words when that is clearer. Indent the body of loops.

COMP3711 Midterm, the plain-words sheet (side 2 of 2): the algorithms, heaps, randomness, lower bounds.

5. The algorithms you may cite as black boxes

Name	What it does	Time	Note
Binary search	find x in a sorted array; variant returns first position ≥ x	O(log n)	halve the range each step
Mergesort	sort by splitting in half and merging	Θ(n log n) always	stable, needs extra array
Merge	combine two sorted lists into one	Θ(n)	two pointers, take the smaller head
Count inversions	pairs i<j with A[i]>A[j]	Θ(n log n)	mergesort; when taking from the right list, add the number left in the left list
Max subarray	largest sum of a contiguous block	O(n log n) D&C, O(n) Kadane	Kadane: cur = max(A[i], cur + A[i])
Karatsuba	multiply two n-bit numbers with 3 half-size multiplications	Θ(n^1.585)	middle term = $(a_1+a_0)(b_1+b_0) - a_1b_1 - a_0b_0$
Strassen	multiply n×n matrices with 7 half-size products	Θ(n^2.81)	plain is 8 products, Θ(n³)
Quicksort	pick pivot, partition (smaller left, bigger right), recurse	expected Θ(n log n) with random pivot, worst Θ(n²)	in place, not stable
Partition	rearrange around pivot x	Θ(n)	returns pivot position
Select(A, k)	k-th smallest without sorting	expected O(n) (random pivot); deterministic O(n) exists	recurse into one side only
Heapsort	build heap, extract max n times	Θ(n log n)	in place, not stable
Counting sort	integers in 1..k: count, prefix sums, place from the back	Θ(n + k)	stable
Radix sort	sort by last digit, then next, … with counting sort	Θ(d(n + k))	linear when numbers < n^c: use base n, c passes
Insertion sort	insert each item into the sorted prefix	Θ(n²) worst, Θ(n + inversions)	best for small or nearly sorted input; stable

6. Heaps: what they are and how to draw them

A **min-heap** is a binary tree filled level by level, left to right, where every parent ≤ its children. Stored in an array: the children of position i are 2i and 2i+1, the parent of i is ⌊i/2⌋. The smallest key is at the root (position 1). A max-heap is the same with ≥.

Insert x: put x in the next free spot (end of array). While x is smaller than its parent, swap them ("bubble up"). O(log n) because the tree has ⌊log n⌋ levels.

Extract-min: remember the root, move the LAST item to the root, then while it is bigger than its smaller child, swap with that child ("sift down"). O(log n).

Build-heap from n items in O(n): for i = ⌊n/2⌋ down to 1, sift A[i] down. Most nodes are near the bottom and sink little: Σ (n/2^(h+1))·h ≤ n.

Drawing an insert sequence (exam question every term, 0.5 off per mistake). Insert 4, 3, 8, 2 into an empty min-heap. Draw the tree after EACH insert and write the array under it.

```
insert 4:   4                    array [4]
insert 3:   3        (3 < 4, swap)          array [3, 4]
           /
          4
insert 8:   3                               array [3, 4, 8]
           / \
          4   8
insert 2:   2        (2 goes under 4, 2 < 4 swap, 2 < 3 sw
           / \                              array [2, 3, 8,
          3   8
         /
        4
```

Check each drawing: is it filled left to right with no gaps? Is every parent ≤ both children?

Heap uses in design questions: "merge k sorted lists" → heap of the k front items, pop the min, push the next from that list, O(N log k). "Keep the k largest" → min-heap of size k. "Top k / smallest k from n" → build heap O(n) then k extracts O(k log n). "Running median" → a max-heap for the low half and a min-heap for the high half.

7. Randomness and expectation (one question every term)

Indicator trick. To find the expected number of things that happen: let X_i = 1 if thing i happens, else 0. Then E[X_i] = Pr[thing i happens], and E[X_1 + … + X_n] = Pr[1] + … + Pr[n]. This is **linearity of expectation** and it works even when the events depend on each other. Write those words on the exam.

Waiting trick. If each try succeeds with probability p, the expected number of tries until the first success is 1/p.

Question	Set-up	Answer
n people get random hats; how many get their own?	Pr[person i gets own] = 1/n, n people	n · 1/n = 1
n items into m boxes; expected pairs sharing a box?	each of C(n,2) pairs shares with prob 1/m	n(n–1)/(2m)
Coupon collector: boxes to get all n coupons?	with i already, each box is new with prob (n–i)/n, wait n/(n–i)	n(1 + ½ + … + 1/n) = Θ(n log n)
Boxes to get the first n/2 coupons?	same, but only the first n/2 stages, each waits ≤ 2	Θ(n): at most n, at least n/2
Distinct coupons after n boxes?	type i never appears with prob $(1 - 1/n)^n$	$n - n(1 - 1/n)^n \approx 0.63n$
Quicksort comparisons?	z_i, z_j compared iff one is the first pivot among z_i..z_j: prob 2/(j–i+1)	Σ = O(n log n)
Randomized Select time?	a "good" pivot (middle half) happens with prob ½ and shrinks the array by ¼	Σ 2·(¾)^i·n = 8n = O(n)

8. Lower bounds (why n log n cannot be beaten by comparisons)

Any algorithm that only compares items is a **decision tree**: each node is a comparison, each leaf an answer. If there are N possible answers the tree needs N leaves, and a binary tree with height h has at most 2^h leaves, so h ≥ log N. Height = number of comparisons in the worst case. Sorting has n! possible answers, so Ω(log n!) = Ω(n log n). **Template:** count the possible outputs, write "the decision tree has at least that many leaves, so its height is at least log of that", simplify. Counting sort and radix sort are allowed to be faster because they never compare two keys.

9. Exam checklist

- Read the target time in the question: it tells you the shape (log² n = log n binary searches; log k = halve k each step; n log k = heap of size k; n = linear scan or Select).
- Write the recurrence even if you cannot solve it. Write the obvious slow solution, then improve it.
- Draw every intermediate heap. Put the array under each tree.
- For expectations: define X_i, give Pr, say "by linearity of expectation", add up. Give exact formulas when asked, not approximations.
- Say the running time of every subroutine you call and add them up.
- Never leave a blank. Partial credit is per step.